ISLS Extension Phase 11 v1.0.0

C29: The Navigator

Spectral-Guided Pattern Space Exploration

TRITON spirals. The mesh triangulates.
The Laplacian reads the topology.
The gradient pulls the spiral toward gold.

Sebastian Klemm

18 March 2026

Abstract

This document specifies `isls-navigator` (C29), the spectral-guided exploration engine for the ISLS Pattern Space.

Three components, each real and tested:

- **TRITON** (`metatron_triton/`): golden-angle spiral search with momentum. Existing Rust crate, tested, eigenständig. Used as path dependency. Not modified.
- **Evolutionary Triangulation** (from SPEC-003r1): k-NN mesh construction with resonance weighting, topology guard (Betti stability), entropy-driven refinement. Reimplemented in Rust on ISLS types. No scipy, no gudhi.
- **C16 Laplacian** (`isls-topology/`): spectral decomposition, eigenvalues, Fiedler vector. Already in ISLS, 12 tests. Used directly.

The integration: TRITON generates candidate points. Each evaluated point becomes a vertex in the mesh. The mesh is triangulated using k-NN + resonance weighting. C16 computes the Laplacian on the mesh. The spectral gradient biases TRITON's momentum toward high-resonance regions. The topology guard rejects mesh updates that break structural invariants. Over time, the mesh becomes a map of the entire Pattern Space — dense where patterns are rich, sparse where they are barren.

What is NOT in this spec:

- No port of DioniceOS MeshHolo (too intertwined with MEF types).
- No port of Spiral Coupling (own state management, own persistence).

- No external libraries for Betti numbers (computed from Euler characteristic on the simplicial complex — ≤ 50 lines).
- No Python. Everything in Rust.

Status: Implementation Specification.

New Crate: C29 (`isls-navigator`).

External: `metatron_triton` (path dependency).

Internal: C16 (`isls-topology`), C17 (`isls-store`), C22 (`isls-artifact-ir`), C23 (`isls-forge`).

Contents

I What Exists and What Must Be Built 5

1 Honest Inventory 5

II Architecture 5

2 The Exploration Loop 5

III Components 7

3 SimplexMesh 7

4 Topology Guard 8

5 Spectral Gradient 9

6 Entropy Control 10

IV TRITON Integration 11

7 Path Dependency 11

8 Evaluator Function 11

V Application: Pattern Space Exploration 11

9 What the Navigator Explores 12

9.1 Mode 1: Forge Config Optimization 12

9.2 Mode 2: Architecture Exploration 12

9.3 Singularity Detection 12

VI Studio Integration 12

VII CLI 13

VIII Acceptance Tests 14

IX Handoff 14

10 What is Built vs. What is Reused 15

11 Test Count **15**

12 Completion Criteria **15**

Part I
What Exists and What Must Be Built

1 Honest Inventory

Component	Exists?	Where	Action
TRITON spiral search	✓	`metatron_triton/`	Path dependency. Do not modify.
TRITON SpectralSignature	✓	`metatron_triton/`	Map to ISLS FiveDState.
TRITON CalibrationStrategy	✓	`metatron_triton/`	Use for Forge auto-tuning.
C16 Laplacian	✓	`isls-topology/`	Use directly.
C16 spectral_gap	✓	`isls-topology/`	Use for mesh quality metric.
C16 Kuramoto	✓	`isls-topology/`	Use for cluster detection.
SimplexMesh (k-NN)	×	—	Build. ∼150 lines.
Resonance weighting	×	—	Build. ∼30 lines.
Topology guard (Betti)	×	—	Build. ∼80 lines.
Entropy control	×	—	Build. ∼40 lines.
Spectral gradient bias	×	—	Build. ∼50 lines.
Store persistence	×	—	Build. ∼60 lines (C17 tables).
DioniceOS MeshHolo	✓	DioniceOS	**Do NOT port.** Wrong types.
DioniceOS SpiralCoupling	✓	DioniceOS	**Do NOT port.** Too intertwined.

Estimated new code: ∼400–500 lines of Rust. Everything else is reuse.

Part II
Architecture

2 The Exploration Loop

The complete loop

```
pub struct Navigator {
    triton: TritonSearch<Box<dyn Fn(&[f64]) -> SpectralSignature>>,
    mesh: SimplexMesh,
    config: NavigatorConfig,
    history: Vec<ExplorationStep>,
}

impl Navigator {
    /// One exploration step
    pub fn step(&mut self) -> ExplorationStep {
        // 1. TRITON generates next candidate point
        let triton_result = self.triton.step();
        let point = triton_result.point.clone();
        let signature = triton_result.signature.clone();
```

```
        // 2. Add point as vertex to the mesh
        let vertex_id = self.mesh.add_vertex(&point, &signature);

        // 3. Connect to k nearest neighbors (k-NN edges)
        let new_edges = self.mesh.connect_knn(vertex_id, self.config.k);

        // 4. Weight edges by resonance
        self.mesh.weight_edges_by_resonance(&new_edges);

        // 5. Form simplices from complete subgraphs (triangles)
        let new_simplices = self.mesh.detect_simplices(&new_edges);

        // 6. Topology guard: check if Betti numbers are stable
        let betti_before = self.mesh.betti_numbers();
        self.mesh.add_simplices(&new_simplices);
        let betti_after = self.mesh.betti_numbers();

        if !self.config.allow_betti_change
            && betti_before != betti_after
            && self.mesh.vertex_count() > self.config.
               min_vertices_for_guard
        {
            // Reject: rollback simplices
            self.mesh.remove_simplices(&new_simplices);
        }

        // 7. Compute Laplacian on the mesh graph
        let laplacian = self.mesh.laplacian_matrix();

        // 8. Spectral gradient: Fiedler vector points toward
        //    the direction of maximum graph connectivity change
        let gradient = self.mesh.spectral_gradient(vertex_id, &laplacian
           );

        // 9. Feed gradient back to TRITON as momentum bias
        let reward = signature.resonance()
            / self.triton.best_signature()
                .map(|s| s.resonance())
                .unwrap_or(1.0);
        self.triton.spiral_mut().update_momentum(&gradient, reward);

        // 10. Entropy check: if local entropy is high, reduce
        //     TRITON radius (exploit). If low, increase (explore).
        let local_entropy = self.mesh.local_entropy(vertex_id);
        self.triton.spiral_mut().adapt_radius(local_entropy);

        // 11. Record step
        let step = ExplorationStep {
            index: self.history.len(),
            point,
            signature,
            vertex_id,
            edges_added: new_edges.len(),
            simplices_added: new_simplices.len(),
            spectral_gap: self.mesh.spectral_gap(&laplacian),
```

```
            local_entropy,
            betti: self.mesh.betti_numbers(),
            best_resonance: self.triton.best_signature()
                .map(|s| s.resonance()).unwrap_or(0.0),
        };
        self.history.push(step.clone());
        step
    }
}
```

Part III
Components

3 SimplexMesh

Definition 3.1 (SimplexMesh).

```
pub struct SimplexMesh {
    vertices: Vec<MeshVertex>,
    edges: Vec<MeshEdge>,
    simplices: Vec<Simplex>,  // triangles (3-vertex), tetrahedra (4-
        vertex)
}

pub struct MeshVertex {
    pub id: usize,
    pub point: Vec<f64>,          // position in parameter space
    pub signature: SpectralSignature,  // (psi, rho, omega)
    pub resonance: f64,           // psi * rho * omega
}

pub struct MeshEdge {
    pub v1: usize,
    pub v2: usize,
    pub weight: f64,              // resonance-based weight
    pub distance: f64,            // Euclidean distance in param space
}

pub struct Simplex {
    pub vertices: Vec<usize>,     // 3 for triangle, 4 for tetrahedron
}
```

Definition 3.2 (k-NN Edge Construction).

```
impl SimplexMesh {
    pub fn connect_knn(&mut self, vertex_id: usize, k: usize) -> Vec<
        MeshEdge> {
        let point = &self.vertices[vertex_id].point;
        let mut distances: Vec<(usize, f64)> = self.vertices.iter()
            .enumerate()
            .filter(|(i, _)| *i != vertex_id)
            .map(|(i, v)| (i, euclidean_distance(point, &v.point)))
            .collect();
        distances.sort_by(|a, b| a.1.partial_cmp(&b.1).unwrap());
```

```
        let mut new_edges = Vec::new();
        for (neighbor_id, dist) in distances.into_iter().take(k) {
            if !self.edge_exists(vertex_id, neighbor_id) {
                let edge = MeshEdge {
                    v1: vertex_id,
                    v2: neighbor_id,
                    weight: 0.0,  // set by weight_edges_by_resonance
                    distance: dist,
                };
                self.edges.push(edge.clone());
                new_edges.push(edge);
            }
        }
        new_edges
    }
}
```

Definition 3.3 (Resonance Weighting)**.** Edge weight is the geometric mean of endpoint resonances, scaled inversely by distance:

```
pub fn weight_edges_by_resonance(&mut self, edges: &[MeshEdge]) {
    for edge in edges {
        let r1 = self.vertices[edge.v1].resonance;
        let r2 = self.vertices[edge.v2].resonance;
        let geo_mean = (r1 * r2).sqrt();
        let weight = geo_mean / (edge.distance + 1e-10);
        // Update the stored edge
        if let Some(e) = self.edges.iter_mut()
            .find(|e| e.v1 == edge.v1 && e.v2 == edge.v2) {
            e.weight = weight;
        }
    }
}
```

This means: edges between two high-resonance points that are close together get the highest weight. The mesh naturally "hardens" around good regions.

4 Topology Guard

Definition 4.1 (Betti Numbers (Lightweight))**.** For a simplicial complex in low dimension (≤ 3):

```
impl SimplexMesh {
    pub fn betti_numbers(&self) -> Vec<usize> {
        let v = self.vertices.len();      // 0-simplices
        let e = self.edges.len();         // 1-simplices
        let f = self.simplices.iter()     // 2-simplices (triangles)
            .filter(|s| s.vertices.len() == 3).count();
        let t = self.simplices.iter()     // 3-simplices (tetrahedra)
            .filter(|s| s.vertices.len() == 4).count();

        // Connected components (b0): use union-find on edges
        let b0 = self.connected_components();

        // Euler characteristic: chi = v - e + f - t
```

```
        let chi = v as i64 - e as i64 + f as i64 - t as i64;

        // b1 = e - v + b0 (for connected graph without higher holes)
        // This is approximate but sufficient for our purpose
        let b1 = (e as i64 - v as i64 + b0 as i64).max(0) as usize;

        // b2 = chi - b0 + b1 (from Euler formula)
        let b2 = (chi - b0 as i64 + b1 as i64).max(0) as usize;

        vec![b0, b1, b2]
    }
}
```

No gudhi. No scipy. 30 lines of Rust.

Definition 4.2 (Topology Guard)**.** A mesh update is accepted only if it does not change the Betti vector beyond a tolerance (default: $\Delta b_i \leq 1$ for each i):

```
pub fn topology_stable(before: &[usize], after: &[usize], tolerance:
   usize) -> bool {
    before.iter().zip(after.iter())
        .all(|(b, a)| (*b as i64 - *a as i64).unsigned_abs() as usize <=
            tolerance)
}
```

This prevents the mesh from developing "holes" (topology changes) due to bad edge additions. The mesh grows monotonically in topological complexity, controlled.

5 Spectral Gradient

Definition 5.1 (Spectral Gradient from Fiedler Vector)**.** The Fiedler vector (second eigenvector of the Laplacian) partitions the graph into two halves. The gradient at a vertex is the direction from the vertex toward the center of the Fiedler-positive cluster (the "good" half):

```
impl SimplexMesh {
    pub fn spectral_gradient(
        &self, vertex_id: usize, laplacian: &Vec<Vec<f64>>
    ) -> Vec<f64> {
        // Use C16's spectral decomposition
        let eigenvalues = eigendecompose(laplacian);
        let fiedler = &eigenvalues.eigenvectors[1]; // 2nd eigenvector

        // Partition: positive Fiedler = cluster A, negative = cluster B
        let vertex_fiedler = fiedler[vertex_id];

        // Centroid of the opposite cluster (where we want to go)
        let target_cluster: Vec<usize> = fiedler.iter().enumerate()
            .filter(|(_, &f)| (f > 0.0) != (vertex_fiedler > 0.0))
            .map(|(i, _)| i)
            .collect();

        if target_cluster.is_empty() {
            return vec![0.0; self.vertices[vertex_id].point.len()];
        }

        // Direction = centroid(target) - vertex.point
```

```
        let dim = self.vertices[vertex_id].point.len();
        let mut centroid = vec![0.0; dim];
        for &tid in &target_cluster {
            for d in 0..dim {
                centroid[d] += self.vertices[tid].point[d];
            }
        }
        for d in 0..dim {
            centroid[d] /= target_cluster.len() as f64;
            centroid[d] -= self.vertices[vertex_id].point[d];
        }

        // Normalize
        let norm: f64 = centroid.iter().map(|x| x * x).sum::<f64>().sqrt
            ();
        if norm > 1e-10 {
            centroid.iter_mut().for_each(|x| *x /= norm);
        }
        centroid
    }
}
```

This is the "invisible string": the Fiedler vector tells TRITON "the other side of the graph has high-resonance points you haven't explored — go there."

6 Entropy Control

Definition 6.1 (Local Entropy)**.** Shannon entropy of the resonance distribution in a vertex's k-neighborhood:

```
impl SimplexMesh {
    pub fn local_entropy(&self, vertex_id: usize) -> f64 {
        let neighbors = self.neighbors_of(vertex_id);
        if neighbors.is_empty() { return 0.0; }

        let resonances: Vec<f64> = neighbors.iter()
            .map(|&n| self.vertices[n].resonance)
            .collect();
        let total: f64 = resonances.iter().sum();
        if total < 1e-10 { return 0.0; }

        let probs: Vec<f64> = resonances.iter()
            .map(|r| r / total)
            .collect();

        -probs.iter()
            .filter(|&&p| p > 0.0)
            .map(|&p| p * p.ln())
            .sum::<f64>()
    }
}
```

High entropy = many similar-quality neighbors = well-explored region → TRITON should explore elsewhere (increase radius).

Low entropy = one dominant point surrounded by poor ones = unexplored gradient → TRITON should exploit here (decrease radius).

Part IV
TRITON Integration

7 Path Dependency

Cargo.toml

```
[dependencies]
metatron-triton = { path = "../../../metatron_triton" }
isls-types      = { path = "../isls-types" }
isls-topology   = { path = "../isls-topology" }
isls-forge      = { path = "../isls-forge" }
isls-store      = { path = "../isls-store" }
```

Requirement 7.1 (TRITON Unchanged)**.** The `metatron_triton` crate is used as-is. No modifications. If the path does not resolve, the navigator compiles with a feature gate `#[cfg(feature = "triton")]` and falls back to a simple random search.

8 Evaluator Function

The evaluator bridges TRITON to the Forge:

```
pub fn forge_evaluator(
    forge: &mut ForgeEngine,
    base_spec: &DecisionSpec,
) -> impl Fn(&[f64]) -> SpectralSignature + '_ {
    move |params: &[f64]| {
        // Map TRITON's [0,1]^n to Forge config
        let mut config = ForgeCalibrationState::from_normalized(params);

        // Run a forge with this config
        let result = forge.forge_with_config(base_spec.clone(), &config)
            ;

        match result {
            Ok(r) => SpectralSignature::new(
                r.quality_score,    // psi: quality
                r.stability_score,  // rho: stability
                r.efficiency_score, // omega: efficiency
            ),
            Err(_) => SpectralSignature::new(0.0, 0.0, 0.0),
        }
    }
}
```

Part V
Application: Pattern Space Exploration

9 What the Navigator Explores

The Navigator has two modes:

9.1 Mode 1: Forge Config Optimization

The parameter space is the Forge's configuration: [pmhd_ticks, opposition, temperature, match_threshold, retries TRITON finds the configuration that maximizes $\psi \times \rho \times \omega$ for a given domain. The mesh shows which configuration regions are "gold" and which are "barren."

9.2 Mode 2: Architecture Exploration

The parameter space is the space of possible architectures. Each dimension encodes an architectural decision (e.g., "include auth module" $= [0, 1]$, "database type" $= [0 = \mathit{sqlite}, 0.5 = \mathit{postgres}, 1 = \mathit{none}]$, "cache layer" $= [0, 1]$). TRITON explores combinations. The evaluator forges each combination and measures quality. The mesh shows which architectural regions produce working software.

9.3 Singularity Detection

A singularity is a vertex whose resonance is $> 2\sigma$ above its neighborhood mean:

```
pub fn detect_singularities(&self) -> Vec<usize> {
    self.vertices.iter().enumerate()
        .filter(|(id, v)| {
            let neighbors = self.neighbors_of(*id);
            if neighbors.len() < 3 { return false; }
            let mean: f64 = neighbors.iter()
                .map(|&n| self.vertices[n].resonance).sum::<f64>()
                / neighbors.len() as f64;
            let variance: f64 = neighbors.iter()
                .map(|&n| (self.vertices[n].resonance - mean).powi(2))
                .sum::<f64>() / neighbors.len() as f64;
            let sigma = variance.sqrt();
            v.resonance > mean + 2.0 * sigma
        })
        .map(|(id, _)| id)
        .collect()
}
```

Singularities are unexplored gold: architectures that score far better than anything nearby. They become candidates for new templates.

Part VI
Studio Integration

New Studio View: Navigator

```
+-----------------------------------------------------+
|  NAVIGATOR                                           |
+-----------------------------------------------------+
|  Mode: [Config Optimization v]                       |
|  Steps: 47/100  Best resonance: 0.823                |
|  [ START ] [ STEP ] [ STOP ]                         |
|                                                      |
|  +-- Mesh Visualization (2D projection) ----------+|
|  |  (canvas: vertices as dots, edges as lines,    ||
|  |   color = resonance, stars = singularities)    ||
|  |                                                 ||
|  |     *           .                               ||
|  |    / \   .---.                                  ||
|  |   .---* /     \                                 ||
|  |        \.     .                                 ||
|  |         '---'                                   ||
|  +-------------------------------------------------+|
|                                                      |
|  +-- Metrics Panel -------------------------------+|
|  | Vertices: 47  Edges: 142  Simplices: 89        ||
|  | Spectral gap: 0.34  Betti: [1, 2, 0]           ||
|  | Singularities: 2                                ||
|  | Local entropy: 0.72 (exploring)                 ||
|  +-------------------------------------------------+|
|                                                      |
|  +-- Singularity Feed ----------------------------+|
|  | ! Vertex 23: resonance 0.91 (2.4 sigma)        ||
|  |   Config: ticks=200 opp=0.3 temp=0.0 thr=0.7  ||
|  |   [Apply to Forge] [Save as Preset]             ||
|  +-------------------------------------------------+|
+-----------------------------------------------------+
```

Part VII
CLI

```
# Start navigator in config optimization mode
isls navigate --mode config --steps 100 --domain rust

# Start navigator in architecture exploration mode
isls navigate --mode architecture --steps 50 --template rest-api

# Show navigator status
isls navigate status

# Apply best found configuration
isls navigate apply-best

# List singularities
isls navigate singularities

# Export mesh as JSON (for visualization)
```

```
isls navigate export-mesh --output mesh.json
```

Part VIII
Acceptance Tests

ID	Name	Procedure
AT-NV1	Mesh vertex add	Add 10 vertices; verify mesh has 10 vertices.
AT-NV2	k-NN edges	Add 5 vertices with k=2; verify each has ≤ 2 edges.
AT-NV3	Resonance weight	Add 2 high-res and 2 low-res vertices; verify high-res edge has higher weight.
AT-NV4	Simplex detection	Add 3 mutually connected vertices; verify 1 triangle simplex formed.
AT-NV5	Betti numbers	Mesh with 4 vertices, 4 edges forming a cycle; verify $b_0 = 1, b_1 = 1$.
AT-NV6	Topology guard	Add simplex that would change Betti; verify rejected when guard active.
AT-NV7	Spectral gradient	Build mesh with two clusters; verify gradient at cluster A points toward cluster B.
AT-NV8	Local entropy	Uniform neighborhood: verify high entropy. One dominant vertex: verify low entropy.
AT-NV9	TRITON integration	Run 20 navigator steps with mock evaluator; verify best resonance improves.
AT-NV10	Singularity detection	Inject one vertex with resonance 3σ above neighbors; verify detected.
AT-NV11	Determinism	Run same seed twice; verify identical mesh and history.
AT-NV12	Fallback without TRITON	Disable triton feature; verify navigator uses random search fallback.

Part IX
Handoff

10 What is Built vs. What is Reused

Component	Source	Est. Lines
SimplexMesh (struct + k-NN + edges)	New	∼150
Resonance weighting	New	∼30
Simplex detection	New	∼40
Betti numbers (lightweight)	New	∼50
Topology guard	New	∼20
Spectral gradient (Fiedler)	New + C16	∼50
Local entropy	New	∼30
Singularity detection	New	∼20
Navigator loop + config	New	∼80
Evaluator bridge to Forge	New	∼40
TRITON spiral + search	Reuse	0 (path dep)
C16 Laplacian + spectral	Reuse	0 (existing)
Total new code		**∼510 lines**

11 Test Count

Phase	New Tests	Cumulative
Phase 1–10	355	355
Phase 11 (C29)	12	$\geq$ 367

12 Completion Criteria

1. SimplexMesh builds mesh from evaluated points with k-NN.
2. Resonance weighting hardens edges around high-quality regions.
3. Betti numbers computed without external libraries.
4. Topology guard rejects destabilizing mesh updates.
5. C16 Laplacian computes spectral gap and Fiedler vector on mesh.
6. Spectral gradient biases TRITON momentum toward unexplored high-resonance regions.
7. Local entropy controls TRITON's explore/exploit radius.
8. Singularity detection flags anomalously high-scoring vertices.
9. TRITON integrated as path dependency, unmodified.
10. Navigator runs 100 steps and converges on a mock evaluator.
11. Total tests $\geq$ 367, zero warnings.
12. **The spiral explores. The mesh remembers. The spectrum guides. The topology guards.**

Generated for ISLS Extension Phase 11 v1.0.0.

∼510 lines of new Rust.
Everything else is reuse: TRITON spirals, C16 analyzes, the mesh bridges.

TRITON is the Goldfisch.
The mesh is the Hamsterball that grows.
The Laplacian is the invisible string.
The topology guard is the immune system.
Singularities are unexplored gold.